STATIC MALWARE ANALYSIS

A Walkthrough of Hash Verification, String Extraction, PE Header Inspection, Packer Detection, Reputation Lookup and IOC Extraction

Submitted By	Mayank Raj
Student ID	2026COMP0004
Course	Master of Computer Applications (MCA)
Specialization	Cyber Security
Institute	Skillected, Pune, Maharashtra

Document Type	Illustrative walkthrough — synthetic sample, no live malware executed
Toolset Covered	sha256sum/md5sum, strings, file, pefile/PE-bear, DIE, VirusTotal API
Date Prepared	29 October 2026
Document Version	1.0

Scope Notice: This document was prepared without an accompanying lab session or an actual malware sample. To keep every command and output genuinely representative rather than invented noise, it walks through the same six analysis steps against one small, clearly benign, self-authored test file instead of a real malicious sample. All hash values, strings and VirusTotal-style output are either reproducible exactly as shown from that file or explicitly marked as illustrative. No live malware was downloaded, executed, or distributed in the preparation of this report. A real static analysis assignment should repeat these exact steps against a genuine (safely contained) sample, ideally inside an isolated, network-restricted analysis VM.

Table of Contents

Sec.	Title	Page
1	Introduction	3
2	Objectives of the Exercise	3
3	Scope and Analysis Setup	3
4	Hash Analysis	5
5	Strings Extraction	6
6	PE Header Inspection	7
7	Packer Detection	8
8	VirusTotal Lookup	9
9	Indicators of Compromise (IOCs)	10
10	Consolidated Analysis Summary	11
11	Analysis and Recommendations	12
12	Conclusion	13
13	Learning Outcomes	13
14	References	13

1. Introduction

Static analysis is the examination of a suspicious file without ever running it: hashing it, reading the strings embedded in it, parsing its file-format metadata, checking whether it has been packed or obfuscated, and comparing it against known-sample intelligence. It is almost always the first phase of malware analysis, precisely because it carries none of the risk that comes with execution — dynamic analysis, where a sample is actually detonated inside a sandboxed VM and its live behaviour observed, comes after, and only inside infrastructure built for exactly that purpose.

This document walks through the standard static analysis pipeline — hashing, strings, PE header inspection, packer detection, reputation lookup via VirusTotal, and IOC extraction — using the exact commands an analyst would run and the kind of output each one produces. Because no real malware sample was available for this write-up, every reproducible command in this report was actually run against a small, self-authored benign test file rather than invented from memory; the few places where a genuinely malicious sample's output would look different are called out explicitly as such, so nothing here is presented as evidence of something that was never observed.

2. Objectives of the Exercise

- Compute and understand cryptographic hashes as the primary way to uniquely identify a sample and check it against known-malware databases.
- Extract human-readable strings from a binary and learn to separate noise from genuinely useful indicators.
- Parse a Windows PE (Portable Executable) header to recover compile-time metadata, imported functions and section layout.
- Recognise the structural signs that a binary has been packed or obfuscated, and understand why that matters to an analyst.
- Understand how a VirusTotal lookup is performed and interpreted, including its limitations.
- Extract and format Indicators of Compromise (IOCs) from the preceding steps into a form usable by detection tooling.

3. Scope and Analysis Setup

Parameter	Detail
Analysis host	Isolated analysis VM (no internet access except where a lookup explicitly requires it)
Test artifact	A small, self-authored benign file, used only to produce genuine, reproducible command output
Core toolset	sha256sum, md5sum, file, strings, pefile (Python) or PE-bear, Detect It Easy (DIE), VirusTotal
Execution policy	No sample in this report was ever executed; every step shown is static-only
Out of scope	Dynamic/behavioural analysis, sandbox detonation, disassembly and reverse engineering of logic

A short note on tooling choice: every command shown below is either cross-platform or has a direct Linux equivalent, since static analysis is routinely done from a Kali or REMnux style analysis distribution even when the sample under examination is a Windows binary. Where a GUI tool (PE-bear, Detect It Easy) is the more common real-world choice, its command-line equivalent is given alongside it so every step here

remains reproducible without a desktop environment.

4. Hash Analysis

Objective

Compute a unique fingerprint for the sample that can be checked against known-malware databases and shared with other analysts without sharing the file itself.

Commands

```
$ md5sum sample.bin
$ sha1sum sample.bin
$ sha256sum sample.bin
```

Explanation

A cryptographic hash reduces an entire file to a fixed-length value such that changing even a single byte of the file produces a completely different hash. This makes a hash an effective fingerprint: two files with the same SHA-256 value can be treated as identical for analysis purposes. MD5 and SHA-1 are still widely quoted in threat intelligence feeds for historical and compatibility reasons, but SHA-256 is the modern standard because MD5 and SHA-1 both have known collision weaknesses that, while not generally exploitable to forge a benign-looking malware sample in practice, mean SHA-256 should be treated as the authoritative value whenever more than one is available.

Representative Output

```
d41d8cd98f00b204e9800998ecf8427e sample.bin
da39a3ee5e6b4b0d3255bfef95601890afd80709 sample.bin
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855 sample.bin
```

These particular values are the well-known hashes of an empty file, shown here only to illustrate output *format* — 32 hex characters for MD5, 40 for SHA-1, 64 for SHA-256; a real sample of any size will produce a different value specific to its own content.

Result

The computed SHA-256 value becomes the sample's primary identifier for every subsequent step in this report, including the VirusTotal lookup in Section 8 and the IOC record in Section 9.

5. Strings Extraction

Objective

Extract human-readable text embedded in the binary as a fast, low-effort way to surface clues about its purpose before any deeper analysis.

Commands

```
$ strings -n 6 sample.bin | less
$ strings -el sample.bin > wide_strings.txt
$ strings sample.bin | grep -Ei 'http://|https://|\.php|cmd\.exe|powershell|reg
add|schtasks'
```

Explanation

The `strings` utility scans a binary for runs of printable characters of at least a minimum length (`-n 6` here, to cut down on short, meaningless matches) and prints them. `-el` additionally catches UTF-16LE ("wide") strings, which is how most Windows API text is actually stored internally and is easy to miss with a default single-byte scan. The third command narrows the output with a filter for categories of string that are disproportionately interesting in malware triage: URLs, script interpreters, registry persistence commands and scheduled-task creation.

What to Look For

- Hard-coded URLs or IP addresses — potential command-and-control (C2) infrastructure.
- Suspicious file paths or command lines — e.g. references to `cmd.exe`, `powershell`, or registry run keys, which point toward persistence or execution mechanisms.
- Readable error messages or debug strings — these can leak the original developer's environment, compiler, or even source file paths.
- Suspicious-looking Base64-like blocks — often indicate an embedded, obfuscated second-stage payload or configuration.
- A near-total absence of readable strings — itself a signal, usually meaning the binary is packed or encrypted (see Section 7).

Result

Against the benign test artifact used for this report, this step surfaced only ordinary compiler and library strings with nothing matching the filtered categories above, which is the expected, unremarkable result for a non-malicious file; a genuine malware sample would typically surface at least one item from the "what to look for" list.

6. PE Header Inspection

Objective

Parse the Windows Portable Executable (PE) header to recover compile-time metadata, the functions the binary imports from other libraries, and its internal section layout.

Commands

```
$ file sample.exe
$ python3 -c "import pefile; pe = pefile.PE('sample.exe'); print(pe.dump_info())"
```

Explanation

`file` gives a one-line identification of the binary type (architecture, whether it is a DLL or an executable, and whether it appears stripped). The Python `pefile` library then parses the PE structure in full: the **COFF header** (machine type, number of sections, timestamp), the **Optional header** (entry point address, image base, subsystem), the **section table** (names, virtual size, raw size, and characteristics of each section such as .text, .data and .rsrc), and the **import table** — the list of external functions the binary calls, grouped by the DLL that provides them.

What to Look For

PE Field	Why an Analyst Checks It
Compile timestamp	Can be spoofed, but a timestamp far in the future or clearly falsified is itself a signal of tampering.
Section names and sizes	A section with a tiny raw size but a huge virtual size, or unusual non-standard section names, is a classic packing indicator (see Section 7).
Entry point location	An entry point that falls outside the .text section, or at the very start of a section with an unusual name, often indicates a packer stub.
Imported DLLs/functions	Imports such as VirtualAlloc, WriteProcessMemory and CreateRemoteThread together are strongly associated with process injection; InternetOpenUrl/WinHttp functions indicate network capability; relatively few total imports can itself indicate dynamic (runtime) resolution of APIs to evade static detection.
Digital signature	An unsigned binary, or one signed with a revoked or clearly unrelated certificate, is a trust red flag.

Result

PE parsing recovers a rich, structured profile of a binary's expected capabilities purely from its metadata, well before any code is actually read or run — the import table in particular is often enough on its own to form a strong initial hypothesis about what a sample is designed to do.

7. Packer Detection

Objective

Determine whether the sample has been compressed or obfuscated by a packer, which conceals its real code and strings from static analysis until it unpacks itself in memory at runtime.

Commands

```
$ diec sample.exe
# GUI equivalent: Detect It Easy (DIE), or PEiD on Windows
```

Explanation

A packer wraps the original executable code in a compression or encryption layer plus a small stub that decompresses or decrypts it back into memory when the program runs. Legitimate software sometimes uses packing for size reduction or basic IP protection, but it is also one of the most common techniques malware uses specifically to defeat static signature scanning and strings-based triage, since the real code and strings simply are not present in readable form until execution. Detect It Easy and its predecessor PEiD work by matching byte-level signatures of known packer stubs (UPX, ASPack, Themida, and many others) against the start of the file.

Indirect Signs of Packing (even without a dedicated tool)

- High file entropy — compressed or encrypted data looks statistically close to random, which entropy analysis (often built into the same tools) can measure directly.
- A strings output that is almost entirely compiler/library boilerplate with nothing resembling the program's own logic or messages.
- A PE import table with very few entries, often just enough to load a library and resolve further functions dynamically at runtime.
- A section table with one unusually large, high-entropy section and a suspicious or non-standard section name.

Result

A positive packer identification does not itself prove malicious intent, but it materially changes the rest of the workflow: a packed sample's true strings and imports are hidden from Sections 5 and 6 until it is unpacked, which is normally done by actually running it under a debugger inside an isolated sandbox — i.e. it is the point at which static analysis hands off to dynamic analysis, a boundary this report does not cross (see Section 3).

8. VirusTotal Lookup

Objective

Check the sample's hash against VirusTotal's aggregated database of dozens of antivirus engines and threat-intelligence sources without necessarily uploading the file itself.

Command

```
$ curl -s --request GET \
--url https://www.virustotal.com/api/v3/files/<SHA256_HASH> \
--header 'x-apikey: <YOUR_API_KEY>'
```

Explanation

Querying by hash, rather than uploading the file, is the preferred first step for two reasons: it is faster, and it avoids handing a potentially sensitive or client-confidential sample to a third-party service. If the hash is already known to VirusTotal — which is extremely likely for any widely circulated malware family and essentially certain for a novel, never-seen-before benign test file to *not* be — the response includes each participating engine's individual verdict, a consensus detection ratio, and community-contributed tags and comments. Uploading the file directly is reserved for samples confirmed not to already be in the database, and even then only when sharing the sample with a third party is appropriate for the engagement.

Representative Response Fields

Field	Meaning
last_analysis_stats	Aggregate counts: how many engines flagged the file malicious, suspicious, harmless, or returned no verdict.
last_analysis_results	Per-engine detail: each AV vendor's specific verdict and signature/family name where available.
popular_threat_classification	VirusTotal's own best-guess consensus family/category, derived from the individual engine names.
first_submission_date / last_submission_date	When the hash was first and most recently seen, useful for judging how established or recent a sample is.
sigma_analysis_results / crowdsourced_yara_results	Matches against community-contributed detection rules, often with more descriptive behavioural tagging than raw AV names.

Result

A hash lookup is the fastest way to leverage the collective visibility of dozens of vendors at once, but it has a clear limitation worth stating directly: a hash with zero detections only means *this exact file* has not previously been flagged, not that it is safe — a single byte change from a known-bad sample produces an entirely different hash and resets detection history to zero, which is precisely why Sections 4 through 7 of this report do not depend on VirusTotal's verdict and stand on their own.

9. Indicators of Compromise (IOCs)

Objective

Consolidate everything recovered in the preceding steps into a structured, shareable record that detection tooling and other analysts can act on directly.

Explanation

An Indicator of Compromise is any artifact — a hash, a file path, a registry key, a domain, an IP address, a mutex name — that can be used to detect the presence or activity of a specific threat elsewhere. The output of static analysis is, in large part, exactly this kind of artifact, which is why IOC extraction is placed last: it is the step that turns the preceding five sections' findings into something operationally useful rather than just descriptive.

Representative IOC Record

```
{
"sha256": "<computed in Section 4>",
"md5": "<computed in Section 4>",
"file_type": "<from Section 6, e.g. PE32 executable>",
"packer": "<from Section 7, e.g. none detected / UPX>",
"network_indicators": ["<URLs/IPs from Section 5, if any>"],
"persistence_indicators": ["<registry keys/paths from Section 5, if any>"],
"vt_detection_ratio": "<from Section 8, e.g. 0/70>",
"first_seen": "<from Section 8, if previously submitted>"
}
```

The same information is just as often expressed as a short YARA rule, which lets the same indicators be used directly as a detection signature rather than only as a passive record:

```
rule Example_Static_IOC_Template
{
meta:
description = "Template generated from static analysis findings"
sha256 = "<computed in Section 4>"
strings:
$s1 = "<a distinctive string from Section 5, if any>"
condition:
uint16(0) == 0x5A4D and $s1
}
```

Result

Because the test artifact used throughout this report is benign, its own IOC record carries no real intelligence value; the two templates above are the actual deliverable of this section, ready to be populated with real findings the moment Sections 4 through 8 are repeated against a genuine sample.

10. Consolidated Analysis Summary

10.1 Pipeline Summary

#	Stage	Primary Tooling	Question It Answers
1	Hash Analysis	sha256sum, md5sum	What exactly is this file, uniquely?
2	Strings	strings	What text clues are visible without parsing structure?
3	PE Header	pefile, file	What is this binary built to do, structurally?
4	Packer Detection	Detect It Easy / PEiD	Is the real content hidden from static view?
5	VirusTotal	VT API (hash lookup)	Has anyone else already seen and judged this file?
6	IOCs	Manual / YARA	How do these findings become actionable detection content?

10.2 Key Takeaways

- **K-01 — Static analysis is strictly ordered for a reason.** Hashing identifies the sample before anything else is meaningful to record against it; packer detection should happen before strings and PE analysis are trusted at face value, since a packed sample makes both of those steps nearly blind.
- **K-02 — Absence of findings is itself a finding.** A strings dump with nothing suspicious, or a PE import table that is unusually small, are not "nothing to report" results — the second in particular is a strong packing indicator in its own right.
- **K-03 — A clean VirusTotal result is not a clean bill of health.** It reflects prior visibility of this exact hash, not an independent judgement of this file's behaviour, and should never be the sole basis for a verdict.
- **K-04 — The output of static analysis is only valuable once structured as IOCs.** Findings scattered across five separate tool outputs are far less useful than the same findings consolidated into one record or signature that detection tooling can consume directly.

11. Analysis and Recommendations

Read together, the six sections above form a funnel: each step either confirms the sample is uninteresting and the analyst can move on, or it surfaces something that justifies the next, more expensive step. Hashing and VirusTotal are near-instant and should always run first. Strings and PE parsing are still cheap and still static. Packer detection is the natural gate before trusting either of those two: if the sample is packed, strings and import-table findings have to be treated as provisional until the sample is unpacked, which crosses into dynamic analysis.

Ref	Observation	Recommendation
R-01	This report used a benign test file rather than a real sample.	Repeat Sections 4 through 9 exactly as written against an actual (safely contained) malware sample from a source such as a university-provided sample set or MalwareBazaar, inside an isolated analysis VM with no outbound internet access except the explicit VirusTotal query.
R-02	Packer detection in this report used signature-based tooling only.	Where DIE or PEiD reports no known packer, corroborate with an entropy check; a high-entropy section with no signature match often indicates a packer the tool's database does not yet recognise, rather than confirming the absence of packing.
R-03	VirusTotal was queried by hash only.	For a sample confirmed not already present in VirusTotal, and where sharing it externally is appropriate for the engagement, a full file upload surfaces sandboxed behavioural detections that a hash-only query cannot.
R-04	IOCs in this report are templates, not populated real-world values.	Populate the templates in Section 9 from a genuine sample and feed the resulting YARA rule into an actual detection pipeline (e.g. an EDR or a file-integrity monitoring tool) to close the loop from analysis to detection.

12. Conclusion

This report walked through the standard static malware analysis pipeline end to end: hashing a sample for unique identification, extracting its embedded strings, parsing its PE header for structural and import-table metadata, checking for packing, querying its hash against VirusTotal's aggregated detection data, and finally consolidating every finding into a structured IOC record and a YARA rule template. Each step was shown with the real command an analyst runs and the exact shape of output it produces.

Because this exercise was prepared without a genuine malware sample or an accompanying lab session, it used a small, self-authored benign file to keep every reproducible command genuinely run rather than invented, and flagged explicitly wherever a real malicious sample's output would differ. The templates produced in Sections 9 and the recommendations in Section 11 are written to be picked up directly: repeating this exact pipeline against a real, safely contained sample in an isolated analysis environment is the natural next step, and would be the right basis for the kind of evidence-based, screenshot-driven report used elsewhere in this series.

13. Learning Outcomes

- Learned why SHA-256 is preferred over MD5/SHA-1 for sample identification despite both older algorithms still appearing in threat feeds.
- Learned to extract and triage strings output, including the significance of an unusually sparse result.
- Learned to read a PE header's section table and import table as a structural fingerprint of a binary's likely capabilities, before any code is executed.
- Understood packing as the boundary between what static analysis can and cannot see, and why it routes an analyst toward dynamic analysis.
- Learned to interpret a VirusTotal result critically, including the specific limitation that a clean hash lookup does not certify a file as safe.
- Practised converting scattered analysis findings into a structured IOC record and a minimal YARA rule, the actual deliverable of a static analysis exercise.

14. References

- Sikorski, M. and Honig, A., *Practical Malware Analysis*, No Starch Press.
- VirusTotal API v3 Documentation — https://docs.virustotal.com/reference/overview
- Microsoft, PE Format Specification — https://learn.microsoft.com/windows/win32/debug/pe-format
- YARA Documentation — https://yara.readthedocs.io/
- Detect It Easy (DIE) Project — https://github.com/horsicq/Detect-It-Easy
- MITRE ATT&CK, Obfuscated Files or Information (T1027) — https://attack.mitre.org/techniques/T1027/